Wi-Fi Coverage Algorithms: Greedy vs Brute Force

GitHib Classroom link

Problem Overview

You are given a set of **demand points** (buildings), each represented as a point in the plane:

```
(x, y)
```

A Wi-Fi access point (AP):

- Is placed at a chosen location
- Covers all demand points within radius **R** meters

Goal: Cover all demand points using as few APs as possible (or maximize coverage with a fixed number of APs).

This is a geometric version of the **Set Cover problem**, which is NP-hard. Exact solutions do not scale.

What You Must Implement

1. A **brute-force baseline** (correct but slow)
2. **At least three distinct greedy algorithms**
3. An experimental comparison
4. A written analysis

Required Data Preparation (Read First)

This lab uses **real campus building data** derived from OpenStreetMap (OSM). Before working on the algorithms in this assignment, you must complete the data preparation step described here:

Preparing Real Campus Data with Overpass Turbo

That page walks you through:

- Downloading building location data from OpenStreetMap using Overpass Turbo
- Exporting the results as CSV
- Cleaning and normalizing the data using a provided C++ tool

The output of that process is a file named:

```
kenyonout.csv
```

This file contains building locations converted into a local, meter-scale coordinate system suitable for algorithmic analysis.

Why this step matters

The Wi-Fi coverage problem you will solve depends directly on the quality and structure of the input data. Using real OpenStreetMap data means:

- The input is large
- The input is messy
- Some assumptions will be imperfect

This is intentional. One of the goals of this lab is to experience how algorithm design interacts with real, imperfect data.

Once you have produced `kenyonout.csv`, return to this page and continue with the algorithmic portion of the assignment.

Baseline: Brute-Force Solver (Reference)

This solver checks all subsets of candidate AP locations. It only works for very small inputs and exists to establish correctness.

```
#include <vector>
#include <cmath>

struct Point {
    double x, y;
};
```

```
bool covers(const Point& ap, const Point& p, double R) {
    double dx = ap.x - p.x;
    double dy = ap.y - p.y;
    return dx*dx + dy*dy <= R*R;
}

int bruteForceMinAPs(const std::vector<Point>& points, double R) {
    int n = points.size();
    int best = n;

    for (int mask = 1; mask < (1 << n); ++mask) {
        int used = __builtin_popcount(mask);
        if (used >= best) continue;

        bool ok = true;
        for (int i = 0; i < n && ok; ++i) {
            bool covered = false;
            for (int j = 0; j < n; ++j) {
                if (mask & (1 << j)) {
                    if (covers(points[j], points[i], R)) {
                        covered = true;
                        break;
                    }
                }
            }
            if (!covered) ok = false;
        }

        if (ok) best = used;
    }

    return best;
}
```

This code is provided for comparison, not performance.

Algorithmic Context: Set Cover and Greedy Approximation

This problem is a direct instance of the **Set Cover problem**, covered in Skiena, *The Algorithm Design Manual*.

In our formulation:

- Each **demand point** (building) is an element to be covered
- Each possible AP placement corresponds to a **set** of demand points it covers
- The goal is to choose as few sets as possible whose union covers all elements

Set Cover is NP-hard. Exact solutions require exponential time in the worst case.

Why Greedy Is Reasonable

Skiena emphasizes that greedy algorithms are often:

- Easy to implement
- Fast enough for large inputs
- Surprisingly effective in practice

The classical greedy set-cover heuristic — choosing the set that covers the largest number of uncovered elements — has a known approximation bound:

```
O(log n)
```

In this lab, you are not proving approximation bounds. You are validating greedy ideas experimentally on real data.

You should think carefully about:

- What your greedy choice is optimizing locally
- Why that local choice might lead to a good global solution
- Where it might fail

Program Interface Specification

Your Wi-Fi planner program must support the following command-line interface. The starter repository will already parse these options.

Required Arguments

```
./wifi_planner INPUT.csv
```

- `INPUT.csv`: cleaned demand points (e.g., `kenyonout.csv`)

Optional Flags

```
--radius R
```

Coverage radius in meters (default provided).

```
--algorithm brute
--algorithm greedy1
--algorithm greedy2
--algorithm greedy3
```

Selects which algorithm to run.

```
--max-aps K
```

Optional cap on number of APs. If reached, report coverage percentage.

```
--timeout-ms T
```

Time limit for brute-force or exploratory solvers. The solver must return the best solution found so far.

Required Output (Text)

Your program must print:

- Number of demand points
- Number of APs placed
- Coverage percentage

- Total runtime

Example:

```
Points: 213
Radius: 35m
Algorithm: greedy2
APs placed: 17
Coverage: 100%
Runtime: 42 ms
```

Greedy Algorithm Requirements

You must implement **at least three different greedy strategies**. They must be meaningfully distinct. Examples:

- Select the AP that covers the most uncovered points
- Place an AP at the densest uncovered region
- Cover the farthest uncovered point first
- Cluster-based or center-of-mass heuristics

You may invent your own strategies.

Experimental Results Expectations

Your write-up must include **quantitative comparisons**. At minimum, include tables like the following.

Small Instance (Brute Force Feasible)

Algorithm	APs Used	Optimal?	Runtime (ms)
Brute Force	6	Yes	850
Greedy 1	6	Yes	2
Greedy 2	7	No	1

Large Instance (Campus Scale)

Algorithm	APs Used	Coverage	Runtime (ms)
Greedy 1	18	100%	48
Greedy 2	15	97%	31
Greedy 3	16	100%	52

Graphs (runtime vs N, APs vs R) are encouraged but not required.

Experimental Evaluation

You must:

- Compare greedy results to brute force on small datasets
- Measure runtime and scalability
- Demonstrate why brute force fails at scale

You are not required to provide a formal proof. You **are required** to provide convincing experimental evidence.

Written Report Requirements

Your write-up must include:

- Description of each greedy algorithm
- Why you expected it to work
- Experimental results and tables/graphs
- Comparison against brute force
- Failure cases and limitations

Grading Rubric

Component	Points
Correct brute-force baseline	20
Three distinct greedy algorithms	30

Experimental evaluation	25
Quality of written analysis	25
Total	**100**

Why We Use a Simplified Model

This model ignores:

- Walls and materials
- Signal attenuation
- Interference and channel overlap

This is intentional. The goal is to isolate **algorithmic reasoning**. Later discussion will examine how the model could be extended without turning this into an RF engineering course.

Admin
